

Module - I

Introduction to Finite Automata

1.1 Introduction

- Finite Automata (FA) are simple **mathematical models** of computation.
- They are used to **recognize patterns** (languages) in strings.
- FA have a **finite number of states** and move from one state to another based on the input symbol.
- They are widely used in:
 - Compiler design (lexical analysis)
 - Design of digital circuits
 - Text processing and pattern matching
 - Modeling of real-life systems

Key Idea:

A finite automaton reads an input string symbol by symbol and changes its state according to a transition function.

1.2 Formal Language and Automata Theory

- A **formal language** is a **set of strings** formed from a **finite set of symbols** called an **alphabet**.
- Finite automata are used to **accept or reject strings** of a language.
- The set of strings accepted by a finite automaton is called the **language** recognized by the automaton.

1.3 Alphabet, String and Language

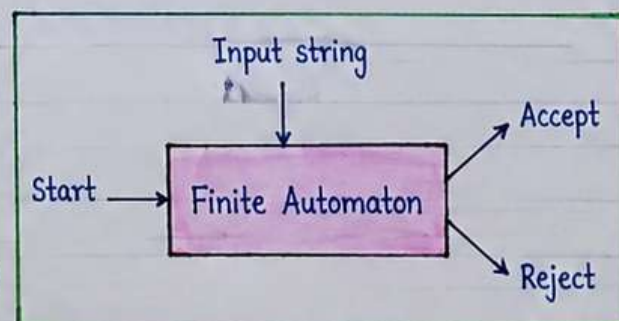
- Alphabet (Σ)**: A finite non-empty set of symbols.
Example: $\Sigma = \{0, 1\}$, $\Sigma = \{a, b, c\}$
- String (word)**: A finite sequence of symbols from Σ .
Example: $w = 0101$, $w = abba$
- Length of a string**: Number of symbols in the string.
Denoted by $|w|$. Example: $|0101| = 4$
- Empty string (ϵ)**: A string of length zero. $\epsilon \in \Sigma^*$.
- Set of all strings (Σ^*)**: The set of all possible finite strings formed from Σ including the empty string.

Examples:

If $\Sigma = \{0, 1\}$
Then $\Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots\}$

1.4 Finite Automaton (FA)

- A Finite Automaton is an abstract machine that reads an input string and determines whether the string is **accepted** or **rejected**.
- It consists of a finite number of states, a start state, a set of accepting states, and a transition function.



1.5 Deterministic Finite Automata (DFA)

1. **Definition** A Deterministic Finite Automaton (DFA) is a finite automaton in which for every state and every input symbol, there is **exactly one next state**.

A DFA is defined as a 5-tuple :

$$M = (Q, \Sigma, \delta, q_0, F) \text{ where,}$$

Q : finite set of states

Σ : finite input alphabet

δ : transition function $\delta : Q \times \Sigma \rightarrow Q$

q_0 : start (initial) state, $q_0 \in Q$

F : set of accepting (final) states, $F \subseteq Q$

Key Properties of DFA

- Finite number of states
- Exactly one transition for each state and input symbol
- No ϵ -transitions
- Deterministic behaviour
- Accepts a string if it reaches a final state after reading the entire input

2. Transition Function

- The transition function δ maps a state and an input symbol to **exactly one next state**.

$$\delta : Q \times \Sigma \rightarrow Q$$

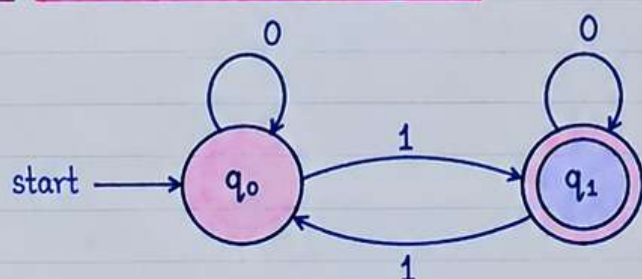
- We write $\delta(q, a) = p$, meaning from state q on input a , the automaton moves to state p .

Example : Consider a DFA

$$M = (\{q_0, q_1\}, \{0, 1\}, \delta, q_0, \{q_1\})$$

δ	0	1
q_0	q_0	q_1
q_1	q_1	q_0

3. State Transition Diagram



Notes :

- Start state is indicated by an incoming arrow.
- Final state is shown by a double circle.
- Each state has exactly one outgoing transition for each input symbol.
- There should be no missing transition.

4. Transition Table

δ	0	1
q_0	q_0	q_1
q_1	q_1	q_0

Relation between Diagram and Table :

- Each arrow in the diagram corresponds to an entry in the table.
- Since it is a DFA, every state has exactly one transition for each input symbol (0 and 1 in this example).

5. Language of a DFA

- The language accepted by a DFA $M = (Q, \Sigma, \delta, q_0, F)$ is the set of all strings $w \in \Sigma^*$ such that the extended transition function $\delta^*(q_0, w) \in F$.

$$L(M) = \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\}$$

- A string is **accepted** if, after reading the entire input, the automaton halts in a final state.

1.6 Examples of DFA

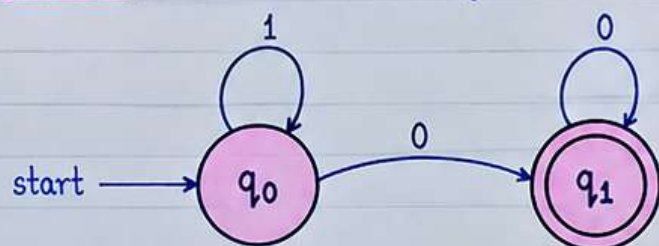
Example 1: DFA for strings over $\{0,1\}$ ending with 0

Step 1: Define states

q_0 : last symbol is not 0 (start state)

q_1 : last symbol is 0 (final state)

Step 2: State transition diagram



Transition Table

δ	0	1
q_0	q_1	q_0
q_1	q_1	q_0

Language

- $L = \{w \in \{0,1\}^* \mid w \text{ ends with } 0\}$
- Examples of accepted strings : 0, 10, 110, 1010
- Examples of rejected strings : ϵ , 1, 11, 101

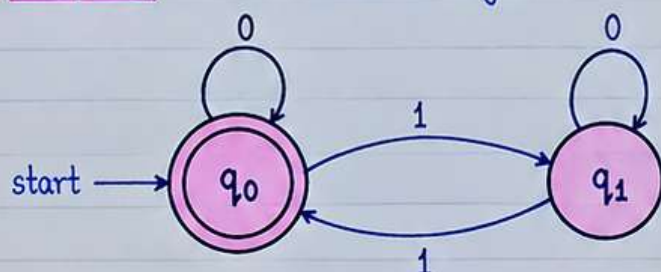
Example 2: DFA for strings with even number of 1's

Step 1: Define states

q_0 : even number of 1's (start and final)

q_1 : odd number of 1's

Step 2: State transition diagram



Transition Table

δ	0	1
q_0	q_0	q_1
q_1	q_1	q_0

Language

- $L = \{w \in \{0,1\}^* \mid \text{number of 1's in } w \text{ is even}\}$
- Examples : ϵ , 0, 00, 11, 1010
- Rejected : 1, 101, 111

Example 3: DFA for strings over $\{0,1\}$ divisible by 3 (binary representation)

Idea :

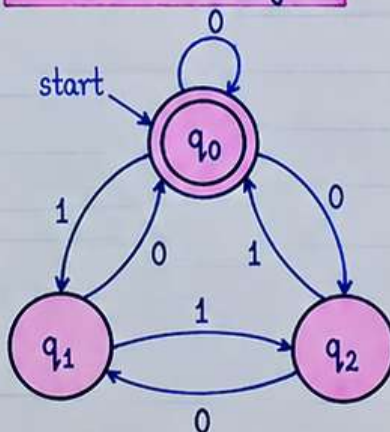
- Keep track of the remainder when the binary number is divided by 3.
- We need 3 states : q_0 (remainder 0), q_1 (remainder 1), q_2 (remainder 2).

Transition function :

$$\delta(q, a) = (2q + a) \bmod 3$$

where $q \in \{0,1,2\}$, $a \in \{0,1\}$.

State transition diagram



Transition Table

δ	0	1
q_0	q_0	q_1
q_1	q_0	q_2
q_2	q_1	q_0

Language

- $L = \{w \in \{0,1\}^* \mid \text{value of } w \text{ in binary is divisible by } 3\}$

1.7 Non-Deterministic Finite Automata (NFA)

1. Definition

- A Non-Deterministic Finite Automaton (NFA) is a finite automaton in which for a state and an input symbol, there may be **zero, one or more next states**.
- It does not have any restriction on the number of transitions for a state and input symbol.
- It accepts a string if there **exists at least one computation path** that leads to a final state.

2. Formal Definition

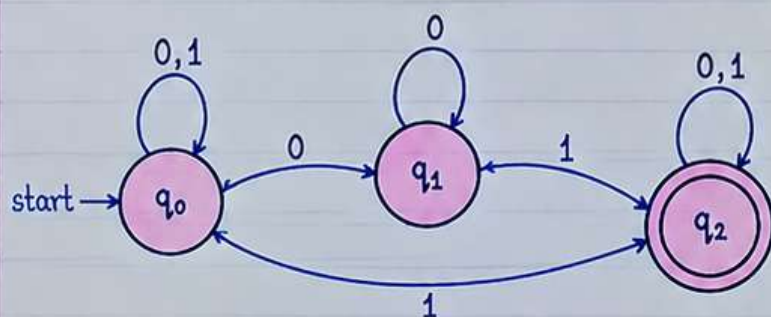
An NFA is a 5-tuple :

$$M = (Q, \Sigma, \delta, q_0, F) \quad \text{where,}$$

- Q : finite set of states
- Σ : finite input alphabet
- δ : transition function $\delta : Q \times \Sigma \rightarrow 2^Q$
- q_0 : start state, $q_0 \in Q$
- F : set of accepting (final) states, $F \subseteq Q$

3. State Transition Diagram

Example NFA : $M = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\})$



Key Differences (DFA vs NFA)

Feature	DFA	NFA
Transition	Exactly one	Zero, one or more
Transition function	$\delta : Q \times \Sigma \rightarrow Q$	$\delta : Q \times \Sigma \rightarrow 2^Q$
Computation	Single path	Multiple paths
Acceptance	One computation must end in final	At least one computation path ends in final
ϵ -transitions	Not allowed	May or may not be allowed (in ϵ -NFA)

Transition Table

State	Input 0	Input 1
q_0	$\{q_0, q_1\}$	$\{q_0, q_2\}$
q_1	$\{q_1\}$	$\{q_2\}$
q_2	$\{q_2\}$	$\{q_2\}$

Notes :

- From q_0 on input 0, NFA can go to either q_0 or q_1 .
- From q_0 on input 1, NFA can go to either q_0 or q_2 .
- A string is accepted if any one path ends in q_2 .

4. Computation of an NFA

- An NFA can have **multiple computation paths** for a given input string.
- If **at least one** of the paths ends in a final state after consuming the entire input string, then the string is **accepted**.
- Otherwise, it is **rejected**.

5. Language of an NFA

The language accepted by an NFA $M = (Q_0, \Sigma, \delta, q_0, F)$ is the set of all strings $w \in \Sigma^*$ for which **there exists at least one computation path** from q_0 to some state in F .

Example : Check acceptance of $w = 01$

- Possible computation paths :
 Path 1 : $q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_2$ (accepted)
 Path 2 : $q_0 \xrightarrow{0} q_1 \xrightarrow{1} q_2$ (accepted)
- Since at least one path reaches a final state, $w = 01$ is **accepted**.

$$L(M) = \{w \in \Sigma^* \mid \hat{\delta}^*(q_0, w) \cap F \neq \emptyset\}$$

where, $\hat{\delta}^*$ is the extended transition function.

1.8 Equivalence of DFA and NFA

- Every DFA is an NFA (a special case).
- Every NFA has an equivalent DFA.
- Thus, DFA and NFA are **equivalent** in power (i.e., they accept the same class of languages).

Key Points :

- $DFA \subseteq NFA$ (trivially)
- For every NFA, we can construct an equivalent DFA using the subset construction method.
- Both accept exactly the regular languages.

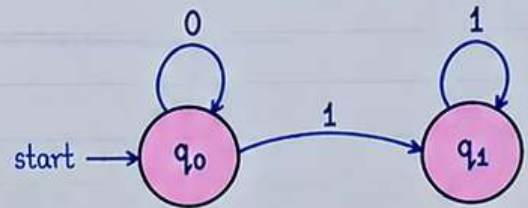
1.9 Conversion : NFA $\rightarrow$ DFA (Subset Construction Method)

Steps :

1. Find the ϵ -closure of the start state (if ϵ -transitions are present; for simple NFA without ϵ , start with $\{q_0\}$).
2. Each state of DFA is a subset of states of NFA.
3. For each subset, find transitions on input symbols using move function.
4. Repeat for all unmarked subsets.
5. Any subset containing a final state of NFA is a final state in DFA.

Example :

Convert the following NFA to DFA.
NFA : $M = (\{q_0, q_1\}, \{0, 1\}, \delta, q_0, \{q_1\})$

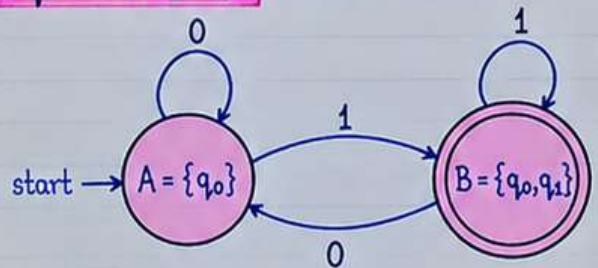


$$\begin{aligned}\delta(q_0, 0) &= \{q_0\}, & \delta(q_0, 1) &= \{q_0, q_1\} \\ \delta(q_1, 1) &= \{q_1\}, & \delta(q_1, 0) &= \emptyset\end{aligned}$$

Subset Construction Table :

DFA State (subset)	Input 0	Input 1	Final?
$\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$	No
$\{q_0, q_1\}$	$\{q_0\}$	$\{q_0, q_1\}$	Yes

Equivalent DFA :



1.10 ϵ -NFA (NFA with ϵ -transitions)

1. Definition :

An ϵ -NFA is an NFA in which the transition function allows ϵ (empty string) as an input symbol, i.e., the automaton can move to a new state without consuming any input symbol.

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$$

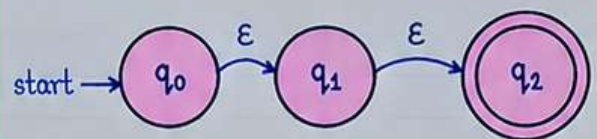
2. ϵ -Closure :

- ϵ -closure of a state q is the set of states reachable from q using only ϵ -transitions (including q itself).
- Denoted by $\epsilon\text{-closure}(q)$.

3. Move Function :

- $\text{move}(q, a) =$ set of states reachable from state q on input symbol a (without ϵ).
- For ϵ -NFA, we use ϵ -closure and move together during conversion to DFA.

Example : ϵ -NFA



Example :

If $\delta(q_0, \epsilon) = \{q_1\}$, $\delta(q_1, \epsilon) = \{q_2\}$,
then $\epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$
 $\epsilon\text{-closure}(q_1) = \{q_1, q_2\}$
 $\epsilon\text{-closure}(q_2) = \{q_2\}$

Notes :

- ϵ -transitions increase the flexibility of the automaton.
- Any ϵ -NFA can be converted to an equivalent NFA without ϵ -transitions.
- ϵ -NFA, NFA and DFA are all equivalent in power.

1.11

Eliminating ϵ -transitions (Conversions of ϵ -NFA to DFA)

1. Idea

- We can convert an ϵ -NFA to an equivalent DFA (without ϵ -transitions) using ϵ -closure.
- For each set of states, we compute its ϵ -closure and then use the move function for each input symbol to construct the DFA using subset construction.

2. Algorithm (Steps)

1. Compute ϵ -closure of every state.
2. Start state of DFA = ϵ -closure(q_0).
3. For each unmarked set of states T and for each input symbol $a \in \Sigma$:
 - $U = \epsilon$ -closure(move(T, a))
 - Add U as a new state if not already present.
4. Repeat until no new states are generated.
5. Any state set containing a final state of ϵ -NFA is a final state in DFA.

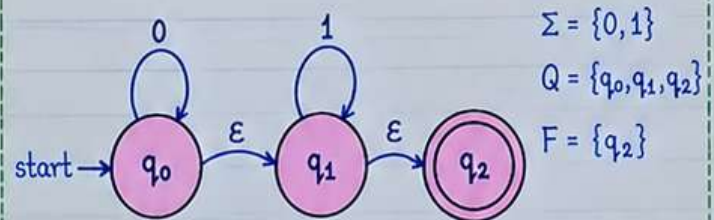
 ϵ -closure (recap)

- ϵ -closure(q) = set of all states reachable from q using only ϵ -transitions (including q itself).
- ϵ -closure(S) for a set $S = \bigcup_{q \in S} \epsilon$ -closure(q).

Move function (recap)

- move(q, a) = set of states reachable from q on input a (without ϵ).
- For a set of states S : move(S, a) = $\bigcup_{q \in S} \text{move}(q, a)$.

Example 1: Convert the following ϵ -NFA to DFA.

Step 1: Compute ϵ -closures

$$\begin{aligned}\epsilon\text{-closure}(q_0) &= \{q_0, q_1, q_2\} \\ \epsilon\text{-closure}(q_1) &= \{q_1, q_2\} \\ \epsilon\text{-closure}(q_2) &= \{q_2\}\end{aligned}$$

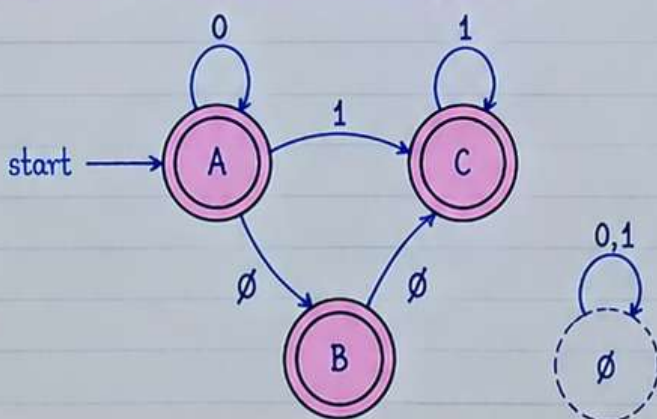
Step 2: Construct DFA states

Start state of DFA = ϵ -closure(q_0)
 $= \{q_0, q_1, q_2\} = A$ (say)
 Let,
 $A = \{q_0, q_1, q_2\}$ (start, final)
 $B = \{q_2\}$ (final)
 $C = \{q_1, q_2\}$ (final)

Step 3: Transition table

DFA State	Input 0	Input 1	Final?
$A = \{q_0, q_1, q_2\}$	A	C	Yes
$C = \{q_1, q_2\}$	$\emptyset$	C	Yes
$B = \{q_2\}$	$\emptyset$	$\emptyset$	Yes
$\emptyset$	$\emptyset$	$\emptyset$	No

Step 4: DFA Transition Diagram



Explanation :

- From A on 0 : move($\{q_0, q_1, q_2\}, 0$) = $\{q_0\}$
 ϵ -closure($\{q_0\}$) = $\{q_0, q_1, q_2\} = A$
- From A on 1 : move($\{q_0, q_1, q_2\}, 1$) = $\{q_1\}$
 ϵ -closure($\{q_1\}$) = $\{q_1, q_2\} = C$
- From C on 0 : move($\{q_1, q_2\}, 0$) = $\emptyset$
- From C on 1 : move($\{q_1, q_2\}, 1$) = $\{q_1\}$
 ϵ -closure($\{q_1\}$) = $\{q_1, q_2\} = C$
- From both C and B on any input, we get $\emptyset$.

Key Takeaways :

- ϵ -transitions can be eliminated by using ϵ -closure.
- The resulting DFA is equivalent (accepts same language).
- ϵ -NFA, NFA and DFA are all equivalent in power.
- This conversion is widely used in compiler design (e.g., regex to DFA).

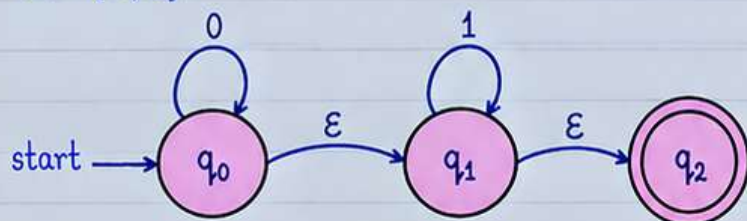
Note :

- The empty set $\emptyset$ is a dead state.
- Any DFA state containing a final state of ϵ -NFA is a final state in DFA.
- Multiple final states may exist in DFA.
- The converted DFA may have more states than the original ϵ -NFA.

1.12 Example : Conversion of ϵ -NFA to DFA

Problem :

Convert the following ϵ -NFA to an equivalent DFA.
 $\Sigma = \{0, 1\}$



Step 1 : Find ϵ -closures

- ϵ -closure(q_0) = $\{q_0, q_1, q_2\}$
 $(q_0 \xrightarrow{\epsilon} q_1 \xrightarrow{\epsilon} q_2)$
- ϵ -closure(q_1) = $\{q_1, q_2\}$
 $(q_1 \xrightarrow{\epsilon} q_2)$
- ϵ -closure(q_2) = $\{q_2\}$
 (no ϵ -transition from q_2)

Step 2 : Compute transitions for each state set

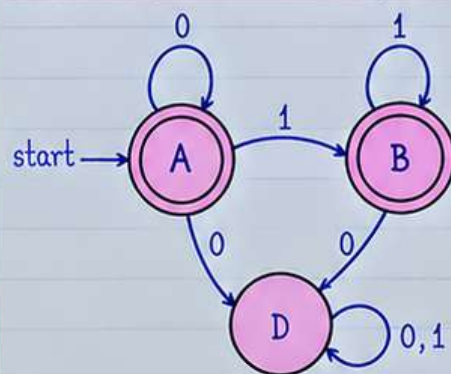
- Start state of DFA = ϵ -closure(q_0) = $\{q_0, q_1, q_2\} = A$

DFA State (set)	On input 0	On input 1	ϵ -closure
$A = \{q_0, q_1, q_2\}$	$\text{move}(A, 0) = \{q_0\}$ ϵ -closure = $\{q_0, q_1, q_2\}$ $= A$	$\text{move}(A, 1) = \{q_1\}$ ϵ -closure = $\{q_1, q_2\}$ $= B$	$\{q_0, q_1, q_2\}$
$B = \{q_1, q_2\}$	$\text{move}(B, 0) = \emptyset$ ϵ -closure = $\emptyset$ $= D$	$\text{move}(B, 1) = \{q_1\}$ ϵ -closure = $\{q_1, q_2\}$ $= B$	$\{q_1, q_2\}$
$D = \emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Step 3 : DFA Transition Table

State	0	1	Final?
$A = \{q_0, q_1, q_2\}$	A	B	Yes
$B = \{q_1, q_2\}$	D	B	Yes
$D = \emptyset$	D	D	No

Step 4 : DFA State Diagram



Observations :

- Each DFA state represents a set of ϵ -NFA states.
- The start state is ϵ -closure of the original start state.
- If the set contains any final state of ϵ -NFA, then it is a final state in DFA.
- The dead state ($\emptyset$) is added to make the DFA complete.

General Algorithm : ϵ -NFA $\rightarrow$ DFA

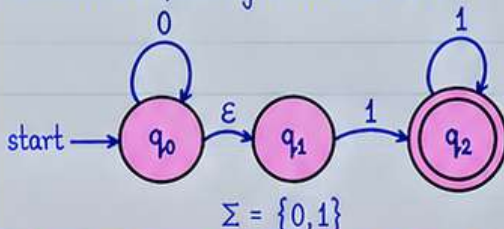
1. Compute ϵ -closure of start state and make it the start state of DFA.
2. For each DFA state (set of NFA states) and each input symbol $a \in \Sigma$:
 - Find $\text{move}(T, a)$ for the set T .
 - Take ϵ -closure of the resulting set.
 - Add it as a new state if not already present.
3. Repeat step 2 for all unmarked states.
4. Mark all states that contain a final state of ϵ -NFA as final states.
5. Add a dead state if required to make DFA complete.

Key Takeaways :

- ϵ -NFA can be converted to an equivalent DFA using ϵ -closure and subset construction.
- The resulting DFA may have up to 2^n states, where n is the number of states in ϵ -NFA.
- Both accept the same language.
- ϵ -transitions make NFA more flexible in design but do not increase expressive power (since they are equivalent to DFA).

Additional Example (self practice) :

Convert the following ϵ -NFA to DFA.



1.13 Minimization of Deterministic Finite Automata (DFA)

1. Idea and Motivation :

- A DFA may have **redundant states** (equivalent states) which can be merged to get a **smaller DFA** that **accepts the same language**.
- Minimization **reduces the number of states** while preserving the language.
- The resulting DFA is called the **minimal DFA** and is **unique** (up to renaming of states).

Equivalent States :

- Two states p and q are equivalent if for every string $w \in \Sigma^*$, both states lead to either accepting or non-accepting states.
- If two states are equivalent, they can be merged into a single state.

Note :

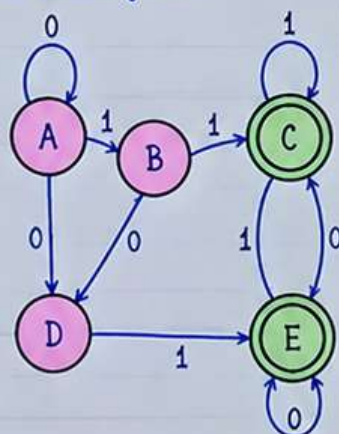
- Minimization applies only to DFA (not directly to NFA).
- First convert NFA/ ϵ -NFA to DFA, then minimize.

2. Algorithm (Table-Filling / Partition Method):

- Remove unreachable states (if any).
- Initially partition the states into two groups :
 - Final states (F)
 - Non-final states (Q-F)
- Refine the partition by splitting groups whose states go to different groups on some input symbol.
- Repeat until no further splitting is possible.
- Each group in the final partition represents one state in the minimal DFA.

3. Example : Minimize the following DFA.

State	0	1	Final?
A	B	C	No
B	B	D	No
C	E	C	Yes
D	B	C	No
E	E	C	Yes



4. Minimization Steps :

Step 1: Initial Partition

Final states : {C, E}

Non-final states : {A, B, D}

$$P_0 = \{\{A, B, D\}, \{C, E\}\}$$

Step 2: Refine {A, B, D}

Check transitions for each state.

State	on 0	on 1	Goes to
A	B	C	({A, B, D}, {C, E})
B	B	D	({A, B, D}, {A, B, D})
D	B	C	({A, B, D}, {C, E})

B differs from A and D (on input 1), so split:

$$P_1 = \{\{A, D\}, \{B\}, \{C, E\}\}$$

Step 3: Refine {A, D}

Check A and D :

State	on 0	on 1	Goes to
A	B	C	({B}, {C, E})
D	B	C	({B}, {C, E})

A and D have same transitions, so they remain together.

$$P_2 = \{\{A, D\}, \{B\}, \{C, E\}\}$$

(No further splitting)

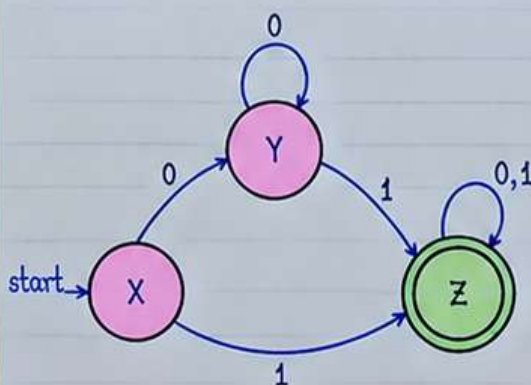
5. Minimized DFA :

Each group becomes one state :

- [A, D] $\rightarrow$ new state X
- [B] $\rightarrow$ new state Y
- [C, E] $\rightarrow$ new state Z (final)

State	0	1	Final?
X([A, D])	Y	Z	No
Y(B)	Y	X	No
Z([C, E])	Z	Z	Yes

6. State Diagram of Minimized DFA :



Observations :

- Original DFA had 5 states.
- Minimized DFA has 3 states.
- States A and D are equivalent.
- States C and E are equivalent.
- Both DFAs accept the same language.

Key Takeaways :

- Minimization removes redundant states.
- The minimal DFA is unique.
- Always remove unreachable states before minimization.

1.14 Alternative Method : Distinguishability Table (Pairwise Method)

1. Idea :

- Mark pairs of states that are distinguishable.
- Two states are distinguishable if one leads to a final state and the other does not, or if for some input string, they lead to distinguishable states.
- Unmarked pairs are equivalent and can be merged.

2. Algorithm (Steps) :

1. List all pairs of states (p, q) with $p \neq q$.
2. Mark pairs where one is final and the other is non-final.
3. Repeat : For each unmarked pair (p, q) , if there exists an input a such that $(\delta(p, a), \delta(q, a))$ is marked, then mark (p, q) .
4. Unmarked pairs are equivalent.
5. Merge equivalent states to get the minimal DFA.

Notes :

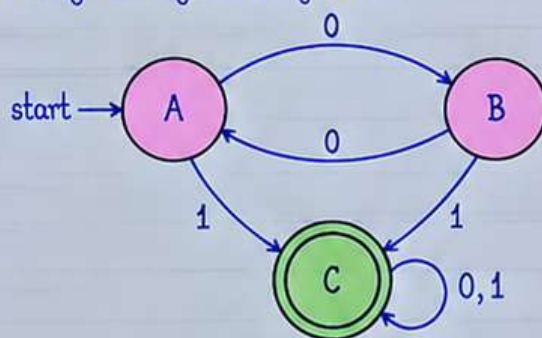
- Applicable only for DFA.
- Often easier for small DFAs.
- Gives the same result as partition method.
- Complexity : $O(n^2)$ pairs to check.

Key Takeaways :

- Both table-filling and partition method give the same minimal DFA.
- Use partition method for larger automata.
- Use distinguishability table for small examples (exam problems).

3. Example : Minimize the following DFA using distinguishability table.

State	0	1	Final?
A	B	C	No
B	A	C	No
C	C	C	Yes



Step 1: Initial Marking

- Mark pairs with one final and one non-final state.

	A	B	C
A	-	-	✓
B		-	✓
C			-

(✓ = marked)

Step 2: Check Transitions

- Remaining unmarked pair: (A, B)
- Check transitions :
On input 0 : $\delta(A, 0) = B$, $\delta(B, 0) = A$
Pair (B, A) is same as (A, B) (unmarked).
- On input 1 : $\delta(A, 1) = C$, $\delta(B, 1) = C$
Pair (C, C) is trivially unmarked.
- So (A, B) remains unmarked.

Equivalent states : $\{A, B\}$

Final Table :

	A	B	C
A	-	-	✓
B		-	✓
C			-

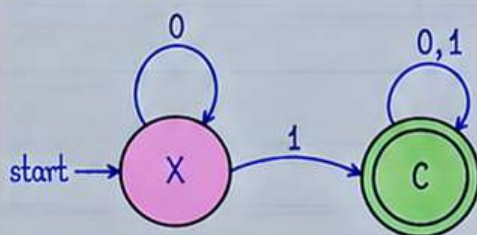
- Unmarked pair $(A, B) \Rightarrow$ equivalent.
- Marked pairs have C (final) $\Rightarrow$ not equivalent.

Step 3: Merge Equivalent States

- Merge A and B into a single state X.
- Keep state C as it is.
- Construct the new transition table.

State	0	1	Final?
X (= {A, B})	X	C	No
C	C	C	Yes

Minimal DFA :



Result :

- Original DFA had 3 states.
- Minimal DFA has 2 states.
- States A and B are equivalent and merged into state X.
- The minimal DFA accepts the same language as the original DFA.

1.15 Designing DFA for a given Language

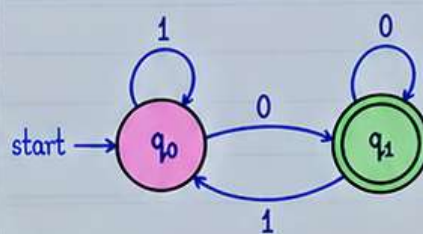
1. General Steps :

1. Understand the language and identify the important conditions (e.g., ends with 0, even number of 1's etc.).
2. Decide the states (what information to remember).
3. Identify the start state.
4. Decide the final states (states that satisfy the language).
5. Define transitions for each input symbol.
6. Draw the state diagram and write the transition table.

Example 1: Strings over $\{0,1\}$ that end with 0

Idea :

- We need to remember the last symbol read.
- Two states are enough :
 $q_0 \rightarrow$ last symbol is not 0 (or start)
 $q_1 \rightarrow$ last symbol is 0 (final)



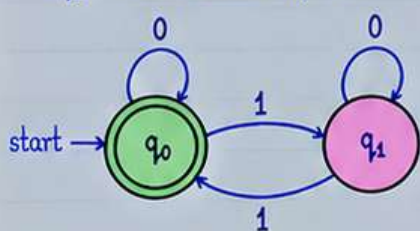
State	0	1
q_0	q_1	q_0
q_1	q_1	q_0

Start state : q_0
Final state : $\{q_1\}$

Example 2: Strings over $\{0,1\}$ with even number of 1's

Idea :

- We need to remember the parity of number of 1's seen so far.
- Two states :
 $q_0 \rightarrow$ even number of 1's (start & final)
 $q_1 \rightarrow$ odd number of 1's



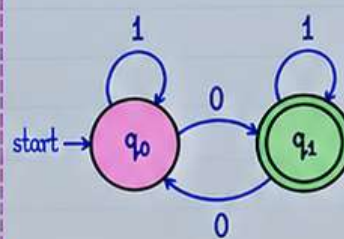
State	0	1
q_0	q_0	q_1
q_1	q_1	q_0

Start state : q_0
Final state : $\{q_0\}$

Example 3: Strings over $\{0,1\}$ with odd number of 0's

Idea :

- Remember the parity of number of 0's.
- $q_0 \rightarrow$ even number of 0's (start)
- $q_1 \rightarrow$ odd number of 0's (final)



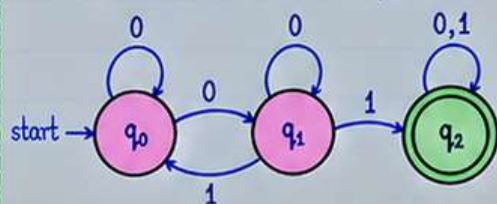
State	0	1
q_0	q_1	q_0
q_1	q_0	q_1

Start state : q_0
Final state : $\{q_1\}$

Example 4: Strings over $\{0,1\}$ that contain substring "01"

Idea :

- We need to remember if we have seen 0 and then 1 afterwards.
- Three states :
 $q_0 \rightarrow$ haven't seen 0 yet
 $q_1 \rightarrow$ seen 0, but not 01 yet
 $q_2 \rightarrow$ seen 01 (final, stay here)



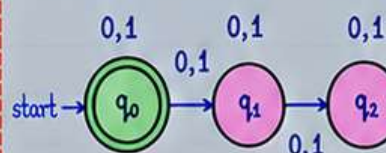
State	0	1
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_2	q_2

Start state : q_0
Final state : $\{q_2\}$

Example 5: Strings over $\{0,1\}$ of length divisible by 3

Idea :

- Use three states to keep count mod 3 of the length.
- $q_0 \rightarrow$ length $\equiv 0 \pmod{3}$ (start & final)
- $q_1 \rightarrow$ length $\equiv 1 \pmod{3}$
- $q_2 \rightarrow$ length $\equiv 2 \pmod{3}$



State	0	1
q_0	q_1	q_1
q_1	q_2	q_2
q_2	q_0	q_0

Start state : q_0
Final state : $\{q_0\}$

Key Takeaways :

- Identify what information needs to be remembered.
- Keep the number of states minimal.
- Draw the state diagram and verify with few test strings.
- Many practical languages can be recognized with small number of states.

Common Types of DFA Design Problems :

- Strings ending with a particular symbol.
- Even / odd number of 0's or 1's.
- Strings containing / not containing a substring.
- Strings of length divisible by k.
- Strings starting with or ending with a substring.
- Combinations of the above conditions.

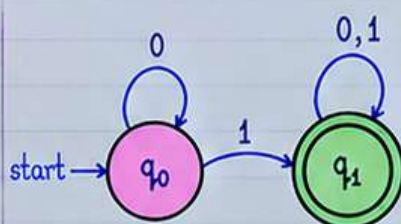
1.16 More Examples of DFA Design

Example 6: Design DFA for strings over $\{0,1\}$ that contain at least one 1.

Steps :

1. Start with a state that has not seen 1 yet.
2. Once we see a 1, move to an accepting state.
3. Stay in accepting state for remaining inputs.
4. Draw state diagram and transition table.

State Diagram :



q_0 : no 1 seen yet (start)
 q_1 : seen at least one 1 (final)

Transition Table :

State	0	1	Final?
q_0	q_0	q_1	No
q_1	q_1	q_1	Yes

Example Strings :

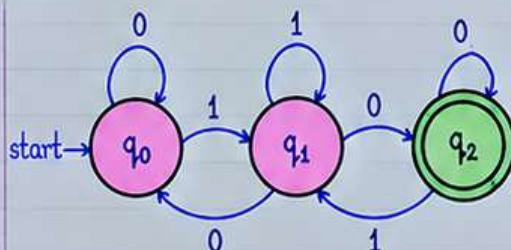
- 0 → Rejected
- 11 → Accepted
- 0101 → Accepted
- 000 → Rejected

Example 7: Design DFA for strings over $\{0,1\}$ that end with 01.

Steps :

1. We need to remember the last two symbols.
2. Use three states to track the suffix.
3. Draw state diagram and transition table.
4. The state q_2 should be accepting (ends with 01).

State Diagram :



q_0 : start state (no useful suffix)
 q_1 : last symbol is 1
 q_2 : last two symbols are 01 (final)

Transition Table :

State	0	1	Final?
q_0	q_0	q_1	No
q_1	q_0	q_1	No
q_2	q_1	q_2	Yes

Example Strings :

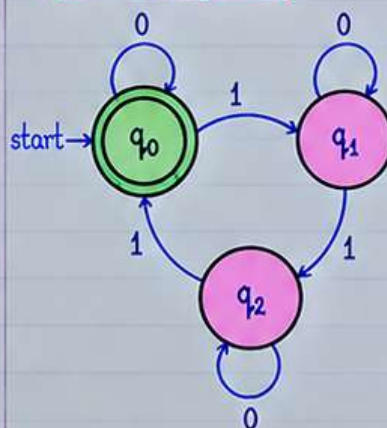
- 01 → Accepted
- 001 → Accepted
- 00 → Rejected
- 1101 → Accepted

Example 8: Design DFA for strings over $\{0,1\}$ in which the number of 1's is divisible by 3.

Idea :

- Use 3 states to keep count of number of 1's mod 3.
- $q_0 \rightarrow \text{count} \equiv 0 \pmod{3}$ (start & final)
- $q_1 \rightarrow \text{count} \equiv 1 \pmod{3}$
- $q_2 \rightarrow \text{count} \equiv 2 \pmod{3}$
- On input 1, move to next state in cycle.
- On input 0, stay in the same state.

State Diagram :



Transition Table :

State	0	1	Final?
q_0	q_0	q_1	Yes
q_1	q_1	q_2	No
q_2	q_2	q_0	No

Example Strings :

- $\epsilon \rightarrow$ Accepted (0 ones)
- 1 → Rejected (1 one)
- 11 → Rejected (2 ones)
- 111 → Accepted (3 ones)
- 1001 → Accepted (3 ones)

Key Takeaways :

- Clearly understand the condition of the language.
- Decide what information needs to be remembered.
- Use minimum number of states.
- Draw state diagram and construct transition table.
- Mark the correct final states.
- Always verify with a few test strings.

Common Mistakes :

- Using more states than necessary (non-minimal DFA).
- Forgetting to handle all possible inputs from every state.
- Incorrectly marking final states.
- Not considering edge cases (empty string, single symbol strings).
- Confusing the condition (e.g., "at least one 1" vs "ends with 1").

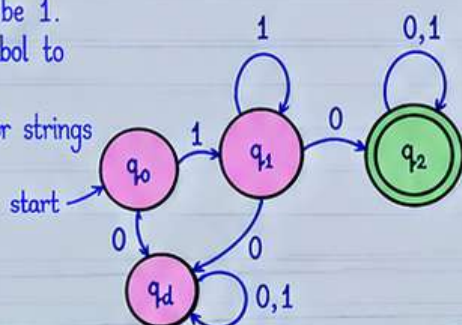
1.17 More Examples and Special DFAs

Example 9: Design DFA for strings over $\{0,1\}$ that start with 1 and end with 0.

Idea:

- First symbol must be 1.
- Track the last symbol to check if it is 0.
- Use a dead state for strings starting with 0.

State Diagram:



Transition Table:

State	0	1	Final?
q_0	q_d	q_1	No
q_1	q_2	q_1	No
q_2	q_2	q_2	Yes
q_d	q_d	q_d	No

Example Strings:

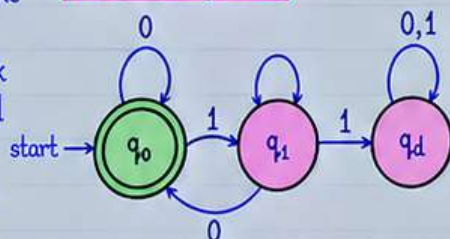
- 10 → Accepted
- 110 → Accepted
- 1010 → Accepted
- 1110 → Accepted
- 1 → Rejected (doesn't end with 0)
- 0 → Rejected (doesn't start with 1)
- 1001 → Rejected (ends with 1)

Example 10: Design DFA for strings over $\{0,1\}$ that do not contain the substring "11".

Idea:

- We can have at most one consecutive 1.
- Use two states to track whether previous symbol was 1 or not.
- If we see 11, move to dead state.

State Diagram:



Transition Table:

State	0	1	Final?
q_0	q_0	q_1	Yes
q_1	q_0	q_d	Yes
q_d	q_d	q_d	No

Example Strings:

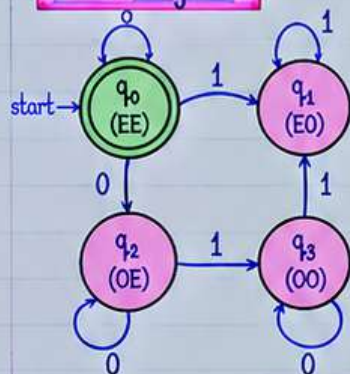
- 0 → Accepted
- 1 → Accepted
- 010 → Accepted
- 1010 → Accepted
- 110 → Rejected
- 0111 → Rejected
- 111 → Rejected

Example 11: Design DFA for even number of 0's and even number of 1's.

Idea:

- We need to track parity of both 0's and 1's.
- Use 4 states: (even, even), (even, odd), (odd, even), (odd, odd).
- Start with (even, even).
- Accept only (even, even).

State Diagram:



Transition Table:

State	0	1	Final?
q_0 (EE)	q_0	q_1	Yes
q_1 (EO)	q_0	q_3	No
q_2 (OE)	q_2	q_3	No
q_3 (OO)	q_1	q_2	No

Example Strings:

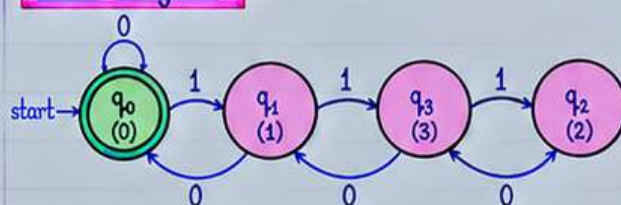
- ϵ → Accepted
- 00 → Accepted
- 11 → Accepted
- 0011 → Accepted
- 0101 → Accepted
- 10 → Rejected
- 01 → Rejected
- 111 → Rejected

Example 12: Design DFA for binary strings divisible by 4 (interpreting the string as a binary number).

Idea:

- Keep remainder modulo 4.
- Use 4 states: 0, 1, 2, 3 (remainder when divided by 4).
- Start at remainder 0 and accept only remainder 0.

State Diagram:



Transition Table:

State	0	1	Final?
q_0 (0)	q_0	q_1	Yes
q_1 (1)	q_3	q_2	No
q_2 (2)	q_1	q_3	No
q_3 (3)	q_2	q_0	No

Key Takeaways:

- DFA design is based on understanding what information must be remembered.
- Use states to keep track of necessary conditions (last symbol, parity, remainder, etc.).
- Use dead state to handle invalid cases.
- Draw state diagram first, then write transition table.
- Always test with a few example strings.

Common Patterns in DFA Design:

- Start / end with specific symbol(s).
- Contain / do not contain a substring.
- Parity of number of 0's or 1's.
- Divisibility by k (use k states).
- Track last symbol, last two symbols.
- Combination of multiple conditions.

1.18 Summary, Important Points and Practice Questions (DFA)

1. Summary of DFA Concepts :

- DFA is a **finite-state machine** that accepts **regular languages**.
- It has a finite number of states and a fixed set of transitions.
- For each state and input symbol, there is **exactly one transition**.
- A state can be **final (accepting)** or non-final (rejecting).
- A string is accepted if the DFA ends in a final state after reading the entire input string.
- DFA can be represented by a **5-tuple** $(Q, \Sigma, \delta, q_0, F)$.
- Equivalent states can be merged to minimize the DFA.
- DFA and **regular expressions** are equivalent in power.

2. Key Takeaways :

- Understand the definition and components of DFA.
- Be clear about how to design DFA for different conditions.
- Use **minimum number of states** whenever possible.
- Always draw state diagram before writing transition table.
- Practice with different types of strings (even/odd, start/end with, contain/avoid substring, divisible by k etc.).
- Learn to minimize DFAs by merging equivalent states.
- DFA is **deterministic**: no multiple transitions for the same input.

3. Common Mistakes :

- Not considering all possible inputs for every state.
- Forgetting to include a dead state when needed.
- Incorrectly marking final states (e.g., accepting before reading entire string).
- Using more states than necessary.
- Confusing "contains" with "ends with" condition.
- Not handling edge cases like empty string (ϵ).
- Wrong transitions (especially for the last symbol).

4. Practice Questions :

Q1 : Design a DFA over $\{0, 1\}$ that accepts all strings having **even number of 0's**.

State Diagram :

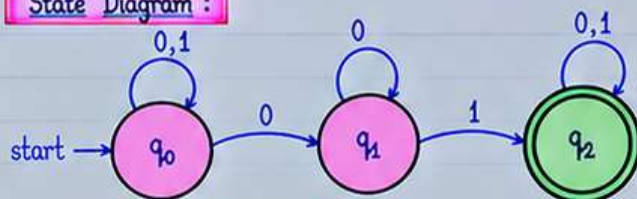


Transition Table :

State	0	1	Final?
q_0	q_1	q_0	Yes
q_1	q_0	q_1	No

Q2 : Design a DFA over $\{0, 1\}$ that accepts all strings **ending with 01**.

State Diagram :

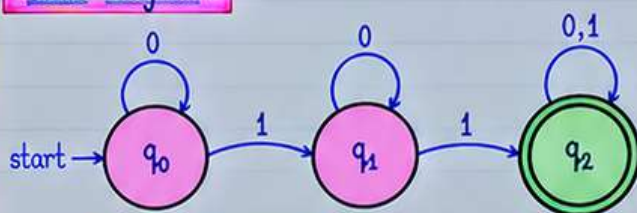


Transition Table :

State	0	1	Final?
q_0	q_0	q_1	No
q_1	q_1	q_2	No
q_2	q_0	q_2	Yes

Q3 : Design a DFA over $\{0, 1\}$ that accepts all strings **containing substring "11"**.

State Diagram :

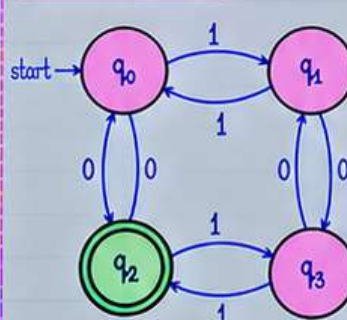


Transition Table :

State	0	1	Final?
q_0	q_0	q_1	No
q_1	q_0	q_2	No
q_2	q_2	q_2	Yes

Q4 : Design a DFA over $\{0, 1\}$ that accepts all strings with **even number of 1's and ending with 0**.

State Diagram :



Transition Table :

State	0	1	Final?
q_0 (E, end1)	q_2	q_1	No
q_1 (O, end1)	q_3	q_0	No
q_2 (E, end0)	q_2	q_3	Yes
q_3 (O, end0)	q_0	q_1	No

State Meaning :

- q_0 : even 1's, ends with 1 (start)
- q_1 : odd 1's, ends with 1
- q_2 : even 1's, ends with 0 (final)
- q_3 : odd 1's, ends with 0

1.19

More Practice Questions and Real Life Perspective (DFA)

Q5 : Design a DFA over $\{0,1\}$ that accepts all strings with **at least two 1's**.

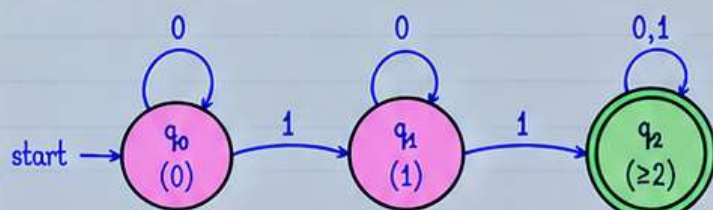
Idea :

- Keep track of how many 1's have been seen (0, 1, or ≥ 2).
- Once we have seen two 1's, stay in the same state.
- Accept strings with count ≥ 2 .

Transition Table :

State	0	1	Final?
q_0	q_0	q_1	No
q_1	q_1	q_2	No
q_2	q_2	q_2	Yes

State Diagram :

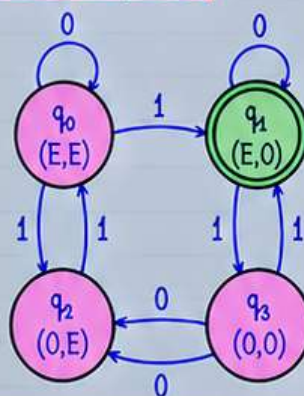


Q6 : Design a DFA over $\{0,1\}$ that accepts all strings with **even number of 0's and odd number of 1's**.

Idea :

- Track parity of 0's (even/odd) and 1's (even/odd).
- Use 4 states : (E,E), (E,O), (O,E), (O,O).
- Accept only (E,O).

State Diagram :



Transition Table :

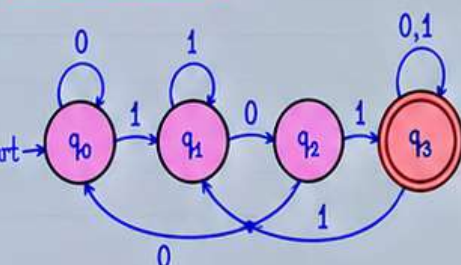
State	0	1	Final?
q_0 (E,E)	q_2	q_1	No
q_1 (E,O)	q_3	q_0	Yes
q_2 (O,E)	q_0	q_3	No
q_3 (O,O)	q_1	q_2	No

Q7 : Design a DFA over $\{0,1\}$ that accepts all strings that **do not contain substring "101"**.

Idea :

- Keep track of the longest suffix matched with the prefix of "101".
- Use 4 states : q_0 (no match), q_1 (seen '1'), q_2 (seen '10'), start q_3 (trap state for '101').
- Reject if we reach trap state.

State Diagram :



Transition Table :

State	0	1	Final?
q_0	q_0	q_1	Yes
q_1	q_0	q_2	Yes
q_2	q_0	q_3	Yes
q_3	q_3	q_3	No

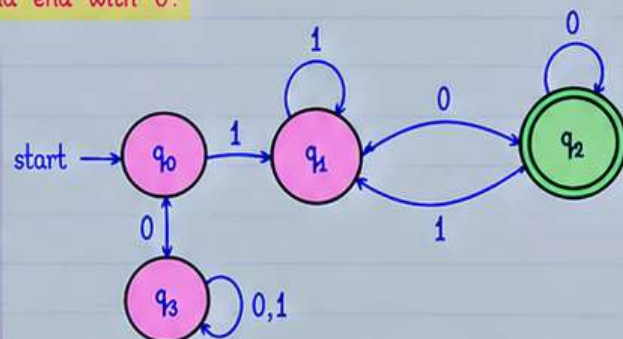
Note :

- q_3 is a trap state (sink state) because once we see "101", we can never remove it.
- All states except q_3 are accepting.

Q8 : Design a DFA over $\{0,1\}$ that accepts all strings that **start with 1 and end with 0**.

Idea :

- First symbol must be 1.
- Then track the last symbol read (0 or 1).
- If the string starts with 0, go to trap state.



Transition Table :

State	0	1	Final?
q_0	q_3	q_1	No
q_1	q_2	q_1	No
q_2	q_2	q_1	Yes
q_3	q_3	q_3	No

Real Life Applications of DFA :

- Lexical analysis in compilers (token recognition).
- Text pattern matching (e.g., checking valid identifiers).
- Digital circuit design (sequence detectors).
- Input validation (e.g., binary numbers divisible by k).
- Designing simple controllers and protocols.

Important Points to Remember :

- DFA is deterministic (exactly one transition).
- One transition for each state and input symbol.
- Use minimum number of states when possible.
- Draw state diagram first, then write transition table.
- Think about what information you need to remember.
- Practice by identifying patterns in different problems.

Practice More :

- Strings with specific suffix/prefix.
- Divisibility by k.
- Even/odd count of symbols.
- Strings with/without a substring.
- Start and end conditions.
- Combine multiple conditions.

1.20

Finite Automata with Output : Introduction, Moore and Mealy Machines

1. Motivation : Why Output Automata ?

- In DFA and NFA, the machine only tells whether a string is accepted or rejected (yes / no).
- In many real systems, we also need an output for each input (e.g., sequence detection, digital circuits, controllers, communication systems).
- For such applications, we use finite automata with output, mainly **Moore** and **Mealy** machines.

2. Finite Automata with Output (Transducers)

- A finite automaton with output is a finite-state machine that **produces an output** while processing an input string.
- It gives an output sequence corresponding to the input string instead of just accepting or rejecting it.
- Two main types :
 - Moore Machine** → Output depends only on the **current state**.
 - Mealy Machine** → Output depends on the current state and current input symbol.

3. Comparison : DFA vs Moore vs Mealy

Feature	DFA	Moore Machine	Mealy Machine
Output	Only Accept/Reject	Produces output sequence	Produces output sequence
Output depends on	—	Current state only	Current state and input symbol
Formal model	5-tuple	6-tuple	6-tuple
Output notation	—	$\lambda(q)$	$\lambda(q, a)$
Output changes	On state change	On state change	Immediately with input
State diagram	Single circle	State with output inside (or separate)	Edge labels with input/output
Used in	Language acceptance	Sequential circuits, controllers	Communication systems, detection

4. Formal Definition

Moore Machine :

- A Moore machine is a 6-tuple $M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$

where,

- Q : finite set of states
- Σ : input alphabet
- Δ : output alphabet
- $\delta : Q \times \Sigma \rightarrow Q$ (transition function)
- $\lambda : Q \rightarrow \Delta$ (output function)
- q_0 : start state

Mealy Machine :

- A Mealy machine is a 6-tuple

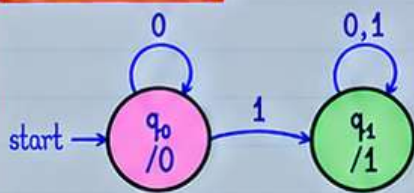
$$M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$$

where,

- Q : finite set of states
- Σ : input alphabet
- Δ : output alphabet
- $\delta : Q \times \Sigma \rightarrow Q$ (transition function)
- $\lambda : Q \times \Sigma \rightarrow \Delta$ (output function)
- q_0 : start state

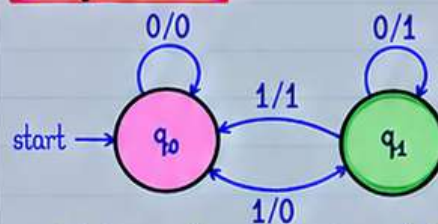
5. Moore vs Mealy Output Timing

Moore Machine :



- Output is written inside the state.
- Output changes only when we move to a new state.
- For a given state, output is fixed.

Mealy Machine :



- Output is written on transitions (edge labels).
- Output depends on current state and input symbol.
- Output can change immediately with input without changing the state (if a self-loop).

6. Example : Sequence Detector (110)

Design a machine that gives output 1 whenever the substring "110" is detected, otherwise 0.

- This can be designed using both Moore and Mealy machines.
- Mealy machine usually needs fewer states.
- Moore machine gives output after reaching the final state.

7. Key Takeaways :

- Moore and Mealy machines are finite automata with output.
- Moore : output depends only on state.
- Mealy : output depends on state and input.
- Both are useful in digital design and real world systems.
- Mealy usually has fewer states, but Moore is simpler to implement.
- We can convert between Moore and Mealy machines (without changing the input-output behavior).

1.21 Moore Machine : Definition, Working and Examples

1. Definition :

- A Moore machine is a finite automaton with output in which the output depends **only** on the **current state** and not on the input symbol.
- The output is **associated with each state**.
- The output changes only when we move to a new state.

2. Formal Model :

A Moore machine is defined as a 6-tuple :

$$M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$$

where,

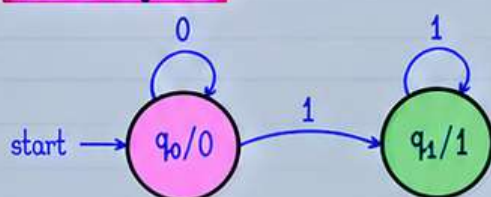
- Q : finite set of states
- Σ : input alphabet
- Δ : output alphabet
- $\delta : Q \times \Sigma \rightarrow Q$ (transition function)
- $\lambda : Q \rightarrow \Delta$ (output function)
- q_0 : start state

3. Characteristics :

- Output depends only on state.
- Output is written inside the state (or as state/output).
- Output changes only after state transition.
- Number of output symbols = Number of states (each state has one output).
- Generally used in sequence detectors, control systems, etc.

4. Moore Machine Representation :

State Diagram :



Transition Table :

Present State (output)	Next State	
	0	1
q_0 (0)	q_0	q_1
q_1 (1)	q_0	q_1

Output Function Table :

State	Output $\lambda(q)$
q_0	0
q_1	1

Note :

- Output is fixed for each state.
- In diagram, we write q /output.

5. Example 1 : Design a Moore machine to detect the substring "01".

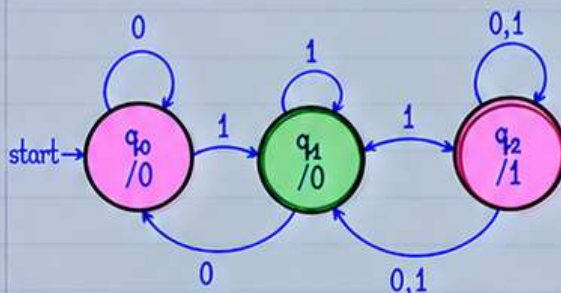
Idea :

- We need to detect when we see 0 followed by 1.
- Use 3 states :
 q_0 : start state (no match),
 q_1 : seen '0'
 q_2 : seen '01' (output 1).

Output :

- $q_0 \rightarrow 0$
- $q_1 \rightarrow 0$
- $q_2 \rightarrow 1$ (when 01 is detected)

State Diagram :



Transition Table :

Present State (output)	Next State	
	0	1
q_0 (0)	q_0	q_1
q_1 (0)	q_0	q_2
q_2 (1)	q_1	q_2

Output Function Table :

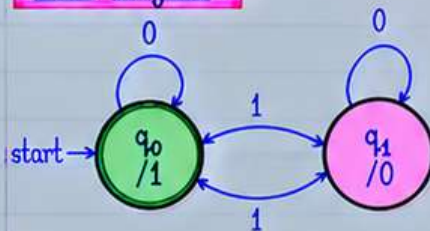
State	Output
q_0	0
q_1	0
q_2	1

6. Example 2 : Design a Moore machine to generate output 1 for every even number of 1's (parity checker).

Idea :

- Use 2 states :
 q_0 : even number of 1's (start)
 q_1 : odd number of 1's
- Output : $q_0 \rightarrow 1, q_1 \rightarrow 0$
- On input 1 $\rightarrow$ toggle state
- On input 0 $\rightarrow$ stay in same state.

State Diagram :



Transition Table :

Present State (output)	Next State	
	0	1
q_0 (1)	q_0	q_1
q_1 (0)	q_1	q_0

Output Function Table :

State	Output
q_0	1
q_1	0

7. Advantages :

- Output is stable (changes only after transition).
- Easy to design and implement.
- Useful in digital circuits and sequential systems.

8. Limitations :

- May require more states compared to Mealy machine.
- Output cannot change immediately with input (since it depends only on state).
- Sometimes less efficient than Mealy machine.

9. Applications :

- Sequence detectors.
- Digital circuit design.
- Control systems.
- Communication systems.
- Parity generators/checkers.

1.22 Mealy Machine : Definition, Working and Examples

1. Definition :

- A Mealy machine is a finite automaton with output in which the **output depends on the current state and the current input symbol**.
- The output is written on the transitions (edges).
- The output changes immediately with input (even without changing the state).
- Generally requires **fewer states** than a Moore machine.

2. Formal Model :

A Mealy machine is defined as a 6-tuple :

$$M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$$

where,

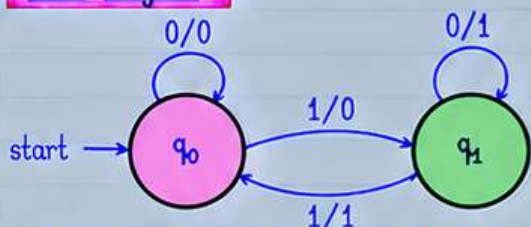
- Q : finite set of states
- Σ : input alphabet
- Δ : output alphabet
- $\delta : Q \times \Sigma \rightarrow Q$ (transition function)
- $\lambda : Q \times \Sigma \rightarrow \Delta$ (output function)
- q_0 : start state

3. Characteristics :

- Output depends on current state and input symbol.
- Output is written on transitions as **input/output**.
- Output changes immediately after reading an input symbol.
- Generally uses fewer states than Moore machine.
- More suitable for real-time systems.
- Used in sequence detectors, communication systems, etc.

4. Mealy Machine Representation :

State Diagram :



Transition Table :

Present State	Input	Next State	Output
q_0	0	q_0	0
q_0	1	q_1	0
q_1	0	q_1	1
q_1	1	q_0	1

Notation :

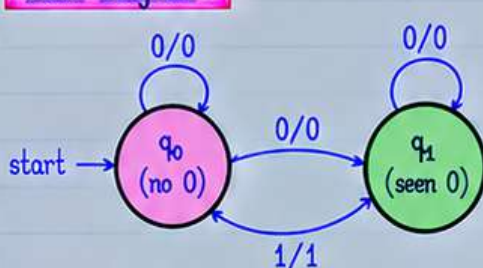
- Edge label : input/output
- If in state q and read input a , we go to state $\delta(q, a)$ and produce output $\lambda(q, a)$.
- Write as : $q \xrightarrow{a/b} p$
(means input a , output b , next state p).

5. Example 1 : Sequence Detector for "01" (Mealy Machine).

Idea :

- We need to detect substring "01". Output 1 when we see 01, otherwise 0.
- Use 2 states :
 q_0 : no match yet
 q_1 : last symbol was 0
- Output 1 only on transition from q_1 to q_0 with input 1.

State Diagram :



Transition Table :

Present State	Input	Next State	Output
q_0	0	q_1	0
q_0	1	q_0	0
q_1	0	q_1	0
q_1	1	q_0	1

Working :

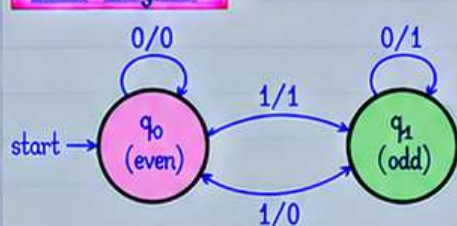
- Input : 1 0 1 0 1 1 0
- Output: 0 0 1 0 1 0 0
- (Output 1 whenever 01 occurs)

6. Example 2 : Parity Checker (Output 1 for odd number of 1's).

Idea :

- Keep track of parity while reading input.
- Two states :
 q_0 : even number of 1's (start)
 q_1 : odd number of 1's
- Output is same as next state parity after reading input.

State Diagram :



Transition Table :

Present State	Input	Next State	Output
q_0	0	q_0	0
q_0	1	q_1	1
q_1	0	q_1	1
q_1	1	q_0	0

7. Advantages :

- Usually requires fewer states than Moore.
- Output changes immediately with input.
- More efficient for many practical systems.
- Useful in sequence detection, communication, control systems.

8. Limitations :

- Output depends on input, so timing may be more complex.
- Not as convenient when output must be stable (use Moore).
- Implementation can be slightly harder in some hardware systems.

9. Key Takeaways :

- Mealy : output depends on state and input.
- Output written on transitions (input/output).
- Generally fewer states than Moore.
- Output changes immediately.
- Useful for sequence detectors.
- Useful Moore and Mealy are equivalent in power (can be converted).

1.23

Conversion between Moore and Mealy Machines

1. Moore to Mealy Conversion :

- A Moore machine can be converted to an equivalent Mealy machine by keeping the **same states and transitions**, and writing the output of the **destination state** on each transition.

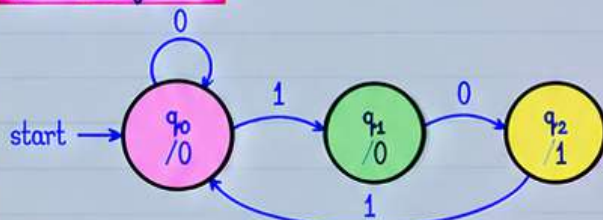
Steps :

- Keep the same set of states and input alphabet.
- For each transition $\delta(q, a) = p$ in the Moore machine, write the output of state p on the transition.
- The start state remains the same.
- The resulting machine will be a Mealy machine.

3. Example : Moore to Mealy Conversion

Given Moore Machine :

State Diagram :

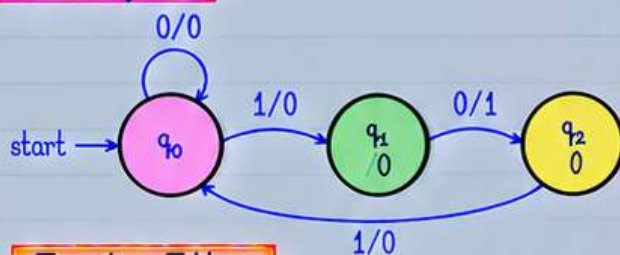


Transition Table :

State	0	1	Output $\lambda(q)$
q_0	q_0	q_1	0
q_1	q_1	q_2	0
q_2	q_0	q_2	1

Converted Mealy Machine :

State Diagram :



Transition Table :

State	0	1	Output
q_0	$q_0/0$	$q_1/0$	
q_1	$q_1/0$	$q_2/0$	
q_2	$q_0/1$	$q_2/1$	

2. Mealy to Moore Conversion :

- A Mealy machine can be converted to an equivalent Moore machine by creating separate states for each **combination of state and output**.

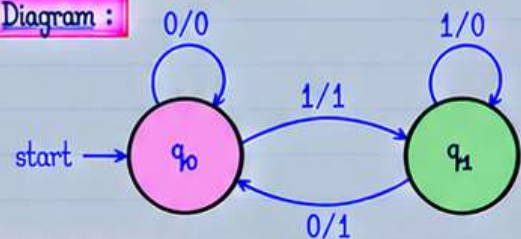
Steps :

- Create a new state for each pair (q, output) that can occur in the Mealy machine.
- The output of each new state is the associated output.
- Add a new start state (if needed) with a dummy output (e.g., -).
- Define transitions based on the Mealy transitions.
- Minimize states if possible (by merging equivalent states).

4. Example : Mealy to Moore Conversion

Given Mealy Machine :

State Diagram :



Transition Table :

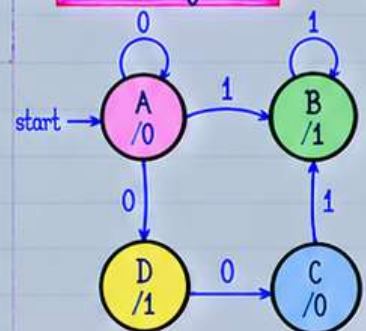
State	0	1
q_0	$q_0/0$	$q_1/1$
q_1	$q_0/1$	$q_1/0$

Converted Moore Machine :

New States :

- $A = (q_0, 0) \rightarrow \text{output } 0$
- $B = (q_1, 1) \rightarrow \text{output } 1$
- $C = (q_1, 0) \rightarrow \text{output } 0$
- $D = (q_0, 1) \rightarrow \text{output } 1$

State Diagram :



Transition Table :

State	0	1	Output
A	A	B	0
B	D	C	1
C	D	C	0
D	A	B	1

5. Important Points :

- Moore $\rightarrow$ Mealy does not increase number of states.
- Mealy $\rightarrow$ Moore may increase number of states.
- Both machines are equivalent in power.
- Moore machine is simpler to implement.
- Mealy machine usually has fewer states.

6. Common Mistakes :

- For Moore $\rightarrow$ Mealy, writing output of current state instead of next state.
- For Mealy $\rightarrow$ Moore, forgetting to create separate states for different outputs.
- Incorrect start state or missing dummy output.
- Not minimizing equivalent states after conversion.

7. Key Takeaways :

- Conversion does not change the language recognized.
- Only the way output is generated changes.
- Moore output depends only on state.
- Mealy output depends on state and input.
- Useful in digital design, controllers, and sequential circuits.

1.24

Module I : Finite Automata (Complete Revision)

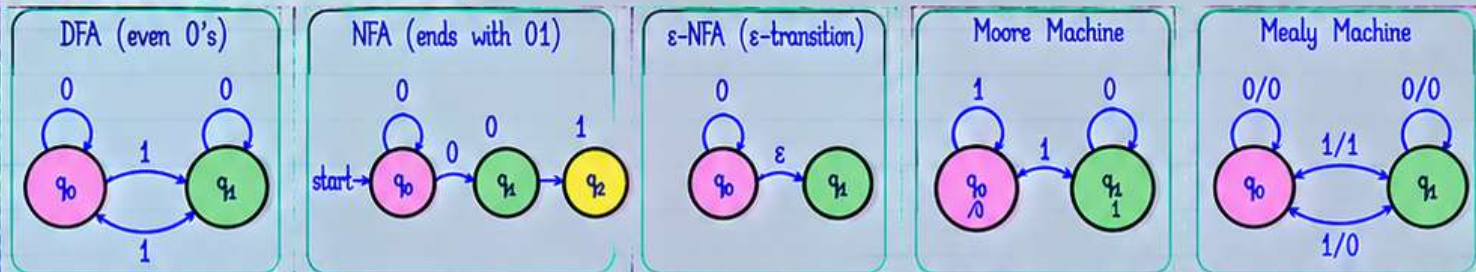
1. Models and Their Comparison :

Feature	DFA	NFA	ϵ -NFA	Moore	Mealy
Formal model	5-tuple	5-tuple	5-tuple	6-tuple	6-tuple
Transition function	$\delta(q, a) \rightarrow q$	$\delta(q, a) \rightarrow$ set of q 's	$\delta(q, a/\epsilon) \rightarrow$ set of q 's	$\delta(q, a) \rightarrow q$	$\delta(q, a) \rightarrow q$
Output	No	No	No	Yes (state based)	Yes (state & input)
ϵ -transitions	Not allowed	Not allowed	Allowed	Not allowed	Not allowed
Deterministic	Yes	No	No	Yes	Yes
Used for	Language acceptance	Language acceptance	Language acceptance	Sequence generator	Sequence generator

2. Formal Definitions (Quick Reference) :

- DFA : $M = (Q, \Sigma, \delta, q_0, F)$
 - NFA : $M = (Q, \Sigma, \delta, q_0, F)$
 - ϵ -NFA : $M = (Q, \Sigma, \delta, q_0, F)$
 - Moore : $M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$
 - Mealy : $M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$
- where,
- Q : finite set of states
 - Σ : input alphabet
 - Δ : output alphabet (for Moore/Mealy)
 - δ : transition function
 - λ : output function
 - q_0 : start state
 - F : set of final states (for DFA/NFA/ ϵ -NFA)

3. Key Diagrams at a Glance :



4. Important Conversions :

NFA $\rightarrow$ DFA	ϵ -NFA $\rightarrow$ DFA	Moore $\rightarrow$ Mealy	Mealy $\rightarrow$ Moore
1 Compute ϵ-closure (if ϵ-NFA). 2 Use subset construction. 3 Each set of states becomes a DFA state. 4 Find final states.	1 Compute ϵ-closure. 2 Remove ϵ-transitions. 3 Convert resulting NFA to DFA (subset construction).	1 Keep same states. 2 For each transition $q \xrightarrow{a} p$ with output $\lambda(p)$, write a/p. 3 Start state remains same.	1 Create new state for each (q, output) pair. 2 Add a new start state if needed. 3 Define transitions. 4 Minimize if possible.

5. DFA Minimization (Steps) :

- 1 Remove unreachable states.
- 2 Partition final and non-final states.
- 3 Refine partitions using distinguishability.
- 4 Merge equivalent states.
- 5 Draw minimized DFA.

6. ϵ -Transition Elimination (Steps) :

- 1 Find ϵ -closure for each state.
- 2 For each input a , compute new transition:

$$\delta'(p, a) = \epsilon\text{-closure} \left(\bigcup_{r \in \epsilon\text{-closure}(p)} \delta(r, a) \right)$$
- 3 Find new final states:
A state is final if its ϵ -closure contains a final state.

7. Common Exam Questions :

- 1 Design DFA for given language.
- 2 Convert NFA/ ϵ -NFA to DFA.
- 3 Minimize a given DFA.
- 4 Design Moore/Mealy machine for a sequence detector.
- 5 Convert Moore to Mealy or Mealy to Moore.
- 6 Find output for a given input string (Moore/Mealy).
- 7 Prove equivalence of DFA and NFA.

8. Important Formulas :

- $\epsilon\text{-closure}(p) = \{q \mid q \text{ reachable from } p \text{ using only } \epsilon\}$
- DFA state count (subset construction) $\leq 2^n$ (n = number of NFA states).
- Number of subsets of an n -state NFA $= 2^n$.
- Minimized DFA has minimum number of equivalent states.

9. Typical Languages and Their DFA :

Language	DFA States (Idea)
Strings ending with 0	2 states
Strings ending with 01	3 states
Even number of 0's	2 states
Odd number of 1's	2 states
Divisible by k (binary)	k states
Strings without substring 101	4 states

10. Key Takeaways :

- DFA, NFA, ϵ -NFA are equivalent in power (same class of languages).
- Moore and Mealy generate output sequences (not for language acceptance).
- Moore output depends only on state.
- Mealy output depends on state and input.
- Conversion between models does not change the input-output behaviour.
- Minimization gives the simplest DFA.
- Finite automata are fundamental for designing digital systems and compilers.